

Dictionaries

Recap

- Tuples and lists are indexable ordered sequences of objects
 - indexed by *position*, 0, 1, 2, ...

Today, we look at a compound type indexed by *meaningful keys* instead of position

Dictionaries

Dictionaries

- *Unordered* collection of **key/value pairs**
 - each key maps to a value
 - keys must be unique and **immutable** (e.g. strings, numbers, tuples)
 - values can be anything, and can repeat
- Denoted by curly braces, `{ }`

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries

```
1  grades = {} # empty dictionary
2  grades = {'Ana': 92, 'Ben': 85, 'Cy': 78} # dictionary literal
3
4  len(grades)
5  grades['Ana']
6  grades['Ben'] = 90
7  grades['Dan'] = 88
8  'Ana' in grades
9  grades['Eli']
```

Dictionaries are mutable

Unlike a tuple, we can assign directly into a key of a dictionary

```
1  grades = {'Ana': 92, 'Ben': 85}
2  grades['Ana'] = 100
3  print(grades)
4
5  grades['Cy'] = 70
6  print(grades)
```

Unlike lists, dictionaries are not indexed by position, 0, 1, 2, ... but by whatever keys you choose

Checking and removing keys

```
1  grades = {'Ana': 92, 'Ben': 85}
2
3  'Ana' in grades
4  'Zoe' in grades
5
6  del grades['Ben']
7  print(grades)
8
9  # dictionary methods
10 grades.get('Zoe')
11 grades.get('Zoe', 0)
```

Use `in` or `.get()` to safely check for a key instead of risking a `KeyError`

Iterating over a dictionary

By default, iterating over a dictionary gives you the *keys*

```
1 grades = {'Ana': 92, 'Ben': 85, 'Cy': 78}
2 for name in grades:
3     print(name, grades[name])
```

You can also iterate over keys and values together with `.items()`

```
1 grades = {'Ana': 92, 'Ben': 85, 'Cy': 78}
2 for name, score in grades.items():
3     print(name, score)
```

- `.keys()` gives just the keys
- `.values()` gives just the values
- `.items()` gives key/value pairs as tuples

Iterating over a dictionary

Both loops on the previous slide produce the exact same result

- the first version looks up each value manually, `grades[name]`
- the second version lets Python hand you each key/value pair directly via `.items()`

Prefer `for key, value in d.items():` whenever you need both the key and the value

- if you only need the keys, `for key in d:` is enough
- if you only need the values, use `for value in d.values():`

Why aren't dictionaries ordered?

Unlike a list, a dictionary does not store its keys by *position*

- when you insert a key, Python runs it through a **hash function**
 - `hash(key)` turns the key into a number
 - that number decides *where* in memory the key/value pair is stored
- the storage location depends on the hash, not on when you added it

This is also why dictionary **keys must be immutable**: if a key could change after being stored, its hash would change too, and Python would no longer be able to find it

Hashing, in a nutshell

A hash function maps a key to a slot number in an internal array

```
1 slot = hash(key) % number_of_slots
```

- `grades['Ana'] = 92` computes `hash('Ana')`, picks a slot, and stores `('Ana', 92)` there
- `grades['Ana']` recomputes `hash('Ana')`, jumps *directly* to that slot, and reads the value
- no need to scan through other keys first
- two different keys can land in the same slot (a **collision**); Python handles this internally, but it's why hashing isn't perfectly even

Hashing, in a nutshell

A toy hash function, just to make the idea concrete:

```
1  def simple_hash(key):
2      total = 0
3      for char in key:
4          total += ord(char)
5      return total
6
7  def slot(key, number_of_slots):
8      return simple_hash(key) % number_of_slots
9
10 print(slot('Ana', 8))
11 print(slot('Ben', 8))
```

- `simple_hash` adds up the character codes of the key - not what Python actually uses, but it shows the pattern: *same key in, same number out*
- `% number_of_slots` squeezes that number down to a valid slot index
- real `hash()` is more complex (and designed to spread keys evenly), but the core idea is identical

Why dictionaries are faster than lists

Suppose we want to look up a value associated with a name

```
1 # with a list of pairs
2 pairs = [('Ana', 92), ('Ben', 85), ('Cy', 78)]
3 for name, score in pairs:
4     if name == 'Cy':
5         print(score)
```

```
1 # with a dictionary
2 grades = {'Ana': 92, 'Ben': 85, 'Cy': 78}
3 print(grades['Cy'])
```

- the list version must check each pair one by one, $O(n)$: the more items, the longer it takes
- the dictionary version jumps straight to the slot using `hash('Cy')`, $O(1)$: roughly the same speed no matter how many keys are stored

This is the key trade-off: dictionaries give up *order* in exchange for *fast lookup by key*

a CRUD app with a dictionary

A dictionary is a natural fit for **C**reate, **R**ead, **U**ppdate, **D**eleate

This structure is called **CRUD** and is the basis for the majority of businesses using the web

Making a crud app using dictionaries

We'll build four small functions, one per CRUD operation, all sharing the same dictionary

```
1 contacts = {}
```

- every function below reads from or writes to this one dictionary
- we'll build them up one at a time, starting with Create

Create: given a dictionary, how would you add a value to it?

```
1 def create(name, phone):  
2     # ...
```

- what should happen if `name` is already a key in `contacts` ?
- what should happen if it isn't?

Create: pick the correct implementation

```
1  # A
2  def create(name, phone):
3      if name in contacts:
4          print(f"{name} already exists")
5      else:
6          contacts[name] = phone
7
8  # B
9  def create(name, phone):
10     contacts[name] = phone
11
12 # C
13 def create(name, phone):
14     if name not in contacts:
15         print(f"{name} already exists")
16     else:
17         contacts[name] = phone
```

Read: given a dictionary, how would you look up a value in it?

```
1 def read(name):  
2     # ...
```

- what should happen if `name` isn't in `contacts` ? Should it crash?

Read: pick the correct implementation

```
1  # A
2  def read(name):
3      return contacts[name]
4
5  # B
6  def read(name):
7      return contacts.get(name, "not found")
8
9  # C
10 def read(name):
11     if name in contacts:
12         return "not found"
13     return contacts[name]
```

Update: given a dictionary, how would you change an existing value?

```
1 def update(name, phone):  
2     # ...
```

- this looks almost identical to Create
- what's the key difference in intent?

Update: pick the correct implementation

```
1  # A
2  def update(name, phone):
3      contacts[name] = phone
4
5  # B
6  def update(name, phone):
7      if name in contacts:
8          contacts[name] = phone
9      else:
10         print(f"{name} not found")
11
12 # C
13 def update(name, phone):
14     if name in contacts:
15         print(f"{name} not found")
16     else:
17         contacts[name] = phone
```

Delete: given a dictionary, how would you remove a value from it?

```
1 def delete(name):  
2     # ...
```

- what Python keyword removes a key/value pair entirely?

Delete: pick the correct implementation

```
1  # A
2  def delete(name):
3      del contacts[name]
4
5  # B
6  def delete(name):
7      contacts[name] = None
8
9  # C
10 def delete(name):
11     if name in contacts:
12         del contacts[name]
13     else:
14         print(f"{name} not found")
```

Using the CRUD scaffold

```
1 create('Ana', '555-1234')
2 create('Ben', '555-5678')
3
4 print(read('Ana'))      # 555-1234
5 print(read('Zoe'))      # not found
6
7 update('Ana', '555-0000')
8 print(read('Ana'))      # 555-0000
9
10 delete('Ben')
11 print(read('Ben'))      # not found
```